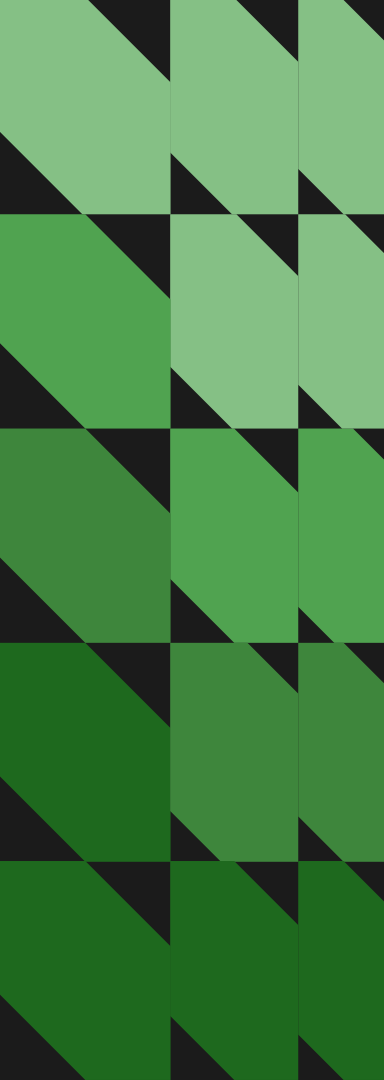


Arquitetura de Software

Instrutor: Ralf Lima



Sobre o módulo

- Carga horária: 10 horas;
- Haverá projeto de acompanhamento (5 horas);
- Intuito:
 - Conhecer as arquiteturas de software para trabalhar com Swift;
 - Dominar o MVVM;
 - Conhecer os complementos do MVVM, através da arquitetura limpa.

O que é arquitetura de software

A arquitetura de software é a estrutura que orienta como um sistema é construído, evolui e se adapta.

Ela define padrões, tecnologias, fluxos de comunicação e camadas de aplicação para que as soluções atendam às necessidades técnicas e de negócio.

Objetivos da arquitetura de software

- Padronização;
- Organização;
- Desempenho;
- Segurança;
- Escalabilidade.

Principais arquiteturas para Swift

1. MV (Model - View)

Uma das arquiteturas mais simples, onde há apenas duas camadas:

- Model: Para criar seus objetos e variáveis;
- View: Para exibir as interfaces gráficas e manipular os dados do Model.

Arquitetura MV



model



view

Principais arquiteturas para Swift

2. VIPER

Mais comum em UIKit, menos usual em SwiftUI por ser muito verbosa.

Camadas da arquitetura Viper

- View (V): Exibe a interface do usuário;
- Interactor (I): É responsável por interagir com APIs ou bancos de dados;
- Presenter (P): Contém a lógica de apresentação. Recebe inputs da View, solicita dados ao Interactor e prepara dados para exibição na View;

Camadas da arquitetura Viper

- Entity (E): Modelos de dados simples usados pela aplicação;
- Router (R): Gerencia a navegação entre as telas.

Arquitectura VIPER



view



interactor



presenter



entity



router

Principais arquiteturas para Swift

3. TCA (The Composable Architecture)

Uma arquitetura baseada em Redux/Elm, ideal para grandes equipes e projetos complexos que exigem forte gerenciamento de estado.

- Estado: Fonte única de verdade;
- Ações: O que pode acontecer;
- Redutores: Lógica que transforma o estado baseado em ações.

Arquitetura TCA



state



action



reducer



store (opcional)



sideEffects (opcional)

Principais arquiteturas para Swift

4. MVI (Model-View-Intent)

É um padrão de projeto para interfaces de usuário que organiza o código em torno de um fluxo de dados unidirecional e cíclico

Características do MVI

- Fluxo Unidirecional: Os dados fluem em uma única direção;
- Estado Único (Model): Centraliza as variáveis e objetos que serão utilizados na aplicação;
- Intenção (Intent): Ações são processadas e atualizam o model.

Arquitectura MVI



model



view



intent

Principais arquiteturas para Swift

5. MVVM (Model-View-ViewModel)

É a arquitetura padrão recomendada para a maioria dos aplicativos SwiftUI.

- Model: Variáveis e objetos;
- ViewModel: Contém a lógica de negócios e estado;
- View: Interface gráfica.

Arquitetura MVVM



model



view



viewModel

Principais arquiteturas para Swift

6. Clean Architecture

Amplamente utilizado com MVVM em projetos maiores.

Ao invés de ter apenas as três camadas base do MVVM, podemos ter outras camadas, deixando o projeto mais padronizado e organizado.

Camadas do Clean Architecture

1. Camada de Apresentação (Presentation Layer)

- Views: Exibem dados e enviam intenções do usuário (o que o usuário quer fazer) para o ViewModel.
- ViewModels/Store: Convertem dados da camada de domínio em modelos adequados para exibição (formatar datas, strings), sem lógica de negócio.
- Routers/Coordinators: Gerenciam a navegação entre telas.

Camadas do Clean Architecture

2. Camada de Domínio (Domain Layer)

- Entities: Modelos de dados de negócio (structs simples);
- Repository Interfaces: Definem contratos (protocols) que a camada de dados deve seguir, sem dizer como o dado é obtido;
- Use Cases (Interactors): Define as regras da aplicação, como por exemplo: Não haver usuários com o mesmo CPF ou a idade não pode ser inferior a zero.

Camadas do Clean Architecture

3. Camada de Dados (Data Layer)

- Repositories Implementation: Decidem se buscam dados da API (remoto) ou banco de dados (local);
- Data Sources (API/Local): Requisições para APIs ou ações na camada de persistência;
- DTOs: Modelos utilizados para mapear dados.

Camadas do Clean Architecture

4. Composition Root (App Level)

O ponto de entrada do app onde a injeção de dependência é feita. Ela conecta todas as camadas (ex: ao criar a view, ela injeta o UseCase, que por sua vez recebe o Repository).

Arquitetura Limpa (Clean Architecture)



presentation



view



viewModel



router



data



repository



source



dto



domain



entity



repository



interactor



composition

Principais arquiteturas para Swift

7. MVVM-C (Model-View-ViewModel Coordinator)

O coordinator, é uma camada para gerenciar as transição entre telas, permitindo que as Views sejam independentes, reutilizáveis e fáceis de testar.

Arquitectura MVVM-C



model



viewModel



view



coordinator

Qual será o nosso foco?

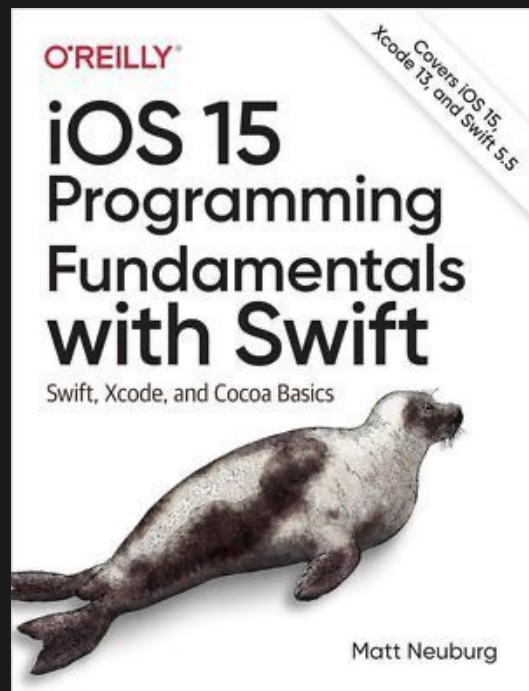
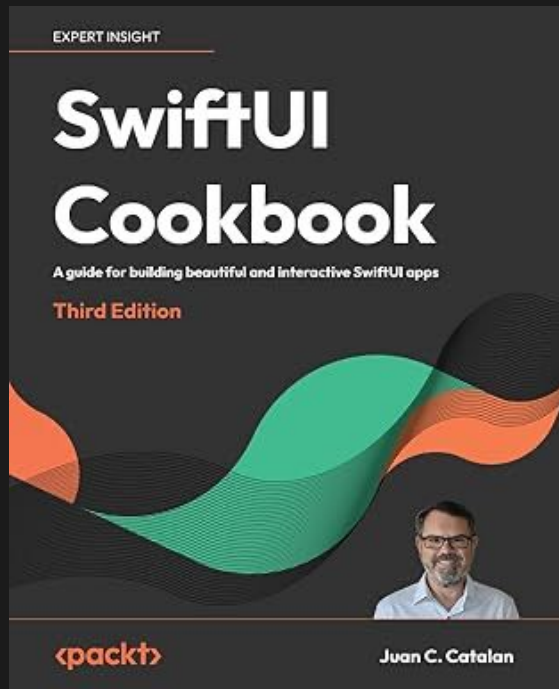


MVVM

Materiais complementares sobre MVVM

- [Adeva](#)
- [Matteo Manfredini](#)
- [Hacking with Swift](#)
- [Radude89](#)
- [Toptal](#)

Indicação de livros



Vamos praticar!

